

ADR-0004: LangChain4j native tools; defer MCP and Google ADK

Status: Accepted **Date:** 2026-06-01 **Deciders:** Eng lead

Context

The agent is a single agent with a fixed, audited tool set (per ADR-0001) plus RAG. We evaluated whether to adopt the **Model Context Protocol (MCP)** and/or **Google's Agent Development Kit (ADK)**. LangChain4j already provides native tool-calling (function calling) and RAG over pgvector, and our tools are deterministic in-process Java services in the same monolith.

Decision

Use **LangChain4j native tool-calling + RAG** for v1. **Do not** adopt MCP or ADK now. Keep tools as thin `@Tool` adapters over plain services so they can later be exposed via MCP or moved under ADK with a thin shim and no rewrite.

Options Considered

Option A: LangChain4j native (chosen)

Pros: In-process calls — fastest, simplest, trivially unit-testable; tightest fit with ADR-0001 ("LLM never calculates"); no extra process/transport/ops. **Cons:** Tools not externally reusable until we add an adapter.

Option B: Add MCP (server/client)

Pros: Standardizes tools across clients; lets us *consume* third-party tool servers. **Cons:** Adds a process boundary, serialization, and ops for zero v1 benefit; works against auditability. **Revisit when:** a licensed data provider ships an MCP server we want to consume, or other internal agents must reuse our tools.

Option C: Adopt Google ADK

Pros: Multi-agent orchestration, A2A, managed Vertex AI Agent Engine deployment. **Cons:** It's an *alternative* to LangChain4j, not an add-on — adopting both means two overlapping frameworks. Our single-agent design doesn't need it. **Revisit when:** we genuinely need multi-agent orchestration or Vertex's managed agent runtime.

Trade-off Analysis

MCP and ADK solve decoupling/multi-agent problems we don't have yet; both would add complexity that fights the determinism/auditability that is the product's core. Native in-process tools are the prudent default.

Consequences

- **Easier:** v1 build, testing, and grounding guarantees.
- **Harder:** external tool reuse needs a future adapter (kept cheap by clean tool boundaries).
- **Revisit:** MCP-as-client for a licensed feed; ADK if multi-agent becomes real.

Action Items

1. ☐ Implement agent tools as thin `@Tool` adapters over `analytics / reconciliation / catalog` services.
2. ☐ Keep the LLM provider behind LangChain4j abstractions (provider-swappable).